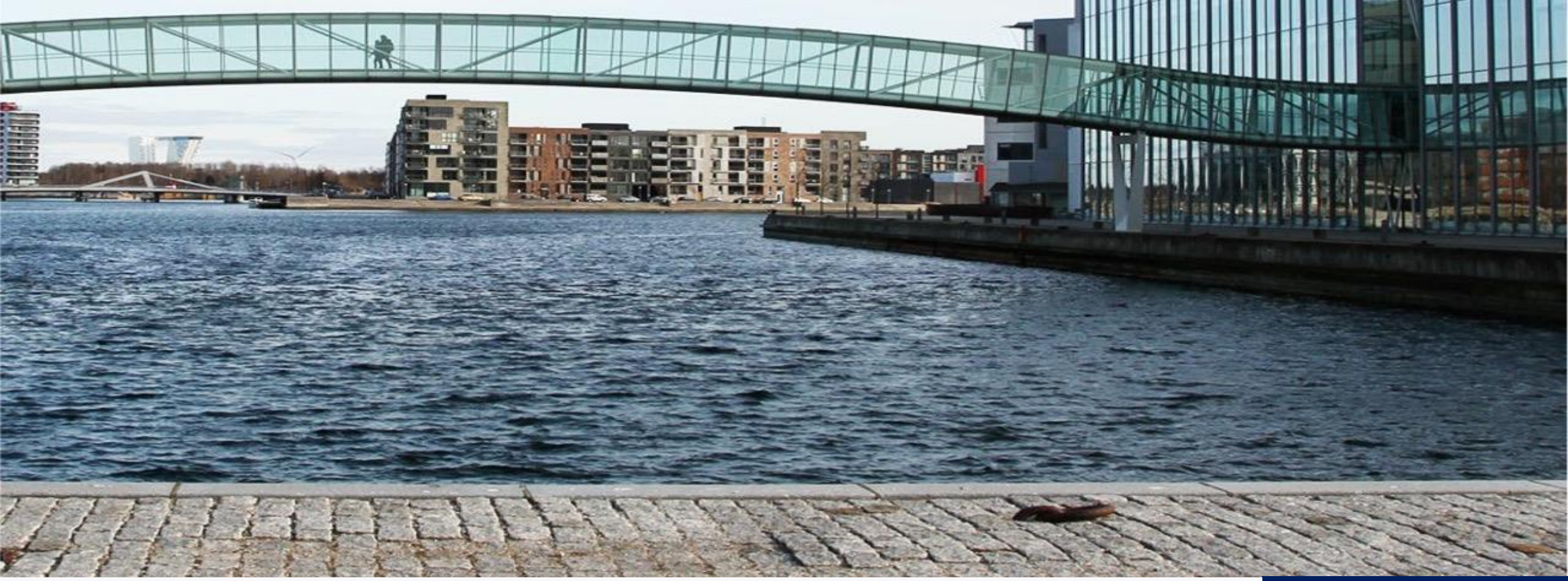


The Mandelbrot Set

Vasilis Ntinas ~ Department of Electronic Systems
vntinas@es.aau.dk

Numerical Scientific Computing ∴ M.Sc. Computer Engineering ∴ Spring 2026

AALBORG UNIVERSITY
COPENHAGEN

What is the Mandelbrot Set

- Famous fractal discovered by Benoit Mandelbrot (1980)
- Mathematical set in complex plane
- Beautiful self-similar patterns
- Infinite detail at any zoom level
- For every point *c* in complex plane C, compute: $z_{i+1} = z_i^2 + c, \quad i = 0, 1, \ldots, I-1$
- where

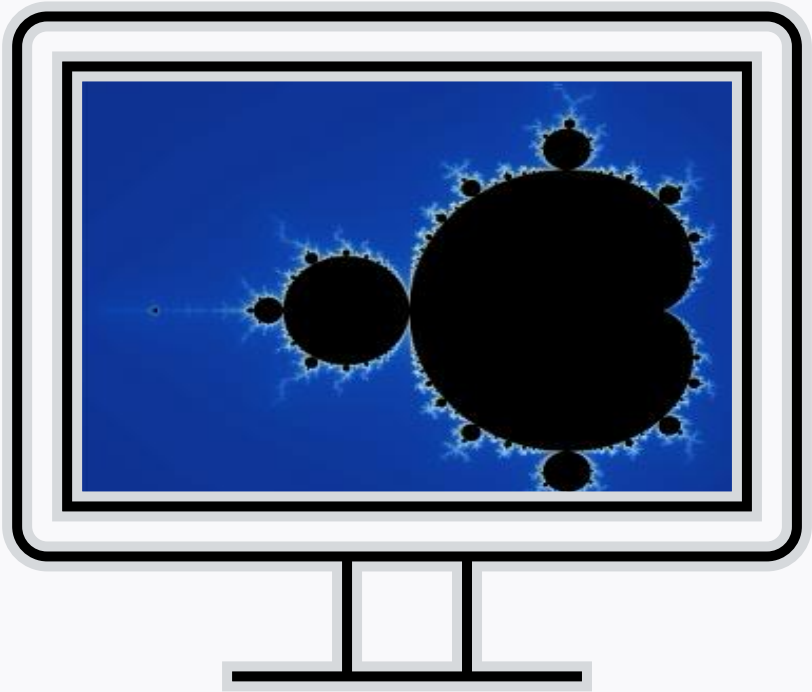
$$\mathbf{C} = \begin{bmatrix} -2.0 & \cdots & 1.0 \\ \vdots & & \vdots \\ -2.0 & \cdots & 1.0 \end{bmatrix} + j \cdot \begin{bmatrix} 1.5 & \cdots & 1.5 \\ \vdots & & \vdots \\ -1.5 & \cdots & -1.5 \end{bmatrix}$$

- For each c we find smallest i where $|z| > T$:

$$\mathfrak{l}(c) = \min \mathbb{T}, \quad \mathbb{T} = \{ i \mid |z_i| > T, i = 1, 2, \ldots, I \} \cup \{I\}$$

- Finally, for each point, we compute:

$$\mathcal{M}(c) = \frac{\mathfrak{l}(c)}{I}$$



The Mandelbrot Set

Algorithm

1. Define region: $x \in [-2,1]$, $y \in [-1.5,1.5]$ (typical view)
2. Create grid of complex numbers c over this region:
   > Generate width evenly spaced x-values from $x_{\min}$ to $x_{\max}$
   > Generate height evenly spaced y-values from $y_{\min}$ to $y_{\max}$
   > Form complex numbers: $c = x + iy$ for each grid point
3. For each point c in grid:
   > Initialize $z_0 = 0$
   > For $n = 0$ to max_iter:
     > Compute $z_{n+1} = z_n^2 + c$
     > If $|z_{n+1}| > 2$: Point escapes! Store n, break to next point
   > If loop completes: Point is in set, store max_iter
4. Return 2D array of iteration counts
5. Visualize with colormap

Key details:

> Threshold $|z| > 2$ guarantees divergence
> Typical: $max_iter = 100$, resolution: 1024×1024 or larger

Embarrassingly Parallel Problems

- In our work, we often do computer simulations:
  - Many repetitions of the same code for statistical confidence
  - Running the same code with different parameters
  - … This can be a tedious process and take a long time.
- Usually, the outer loop of our simulation can be parallelized.

Why Mandelbrot?

- Computationally intensive
  - Naive version is slow → need optimization!
  - Obvious performance improvements
  - Easy to measure speedup
- Embarrassingly parallel
  - Each pixel independent
  - Perfect for multiprocessing, GPU, and distributed computing
  - Can distribute across cluster
- Visual feedback
  - Immediately see if code works (pretty picture!)
  - Easy to debug (if image wrong, something's wrong)
  - Motivating to see results